

Problem2

May 3, 2026

0.1 2.2 Shortest path through a grid

(a)

Doing DP backward (B to A) we compute at each node the optimal cost-to-go (J^* in the figure) and store the optimal edge to follow, represented as a blue arrow. Finally, the optimal path from A to B is computed by always choosing the optimal edge and is highlighted in red. It corresponds to: **up, up, down, up, down, down**.

```
[1]: from IPython.display import Image, display
display(Image(filename='/Users/erwinpoussi/Downloads/NBMetadataCache 2.png',
↪width=700))
```

on a grid. Consider the shortest path problem in a grid. The grid is a diamond shape with nodes at the intersections. The cost to go from a node to the right and to the left is given by the edge weights. The cost to go from a node to the right and to the left is given by the edge weights. Given a two-dimensional state space, we will use dynamic programming to find the shortest path from node A to node B.

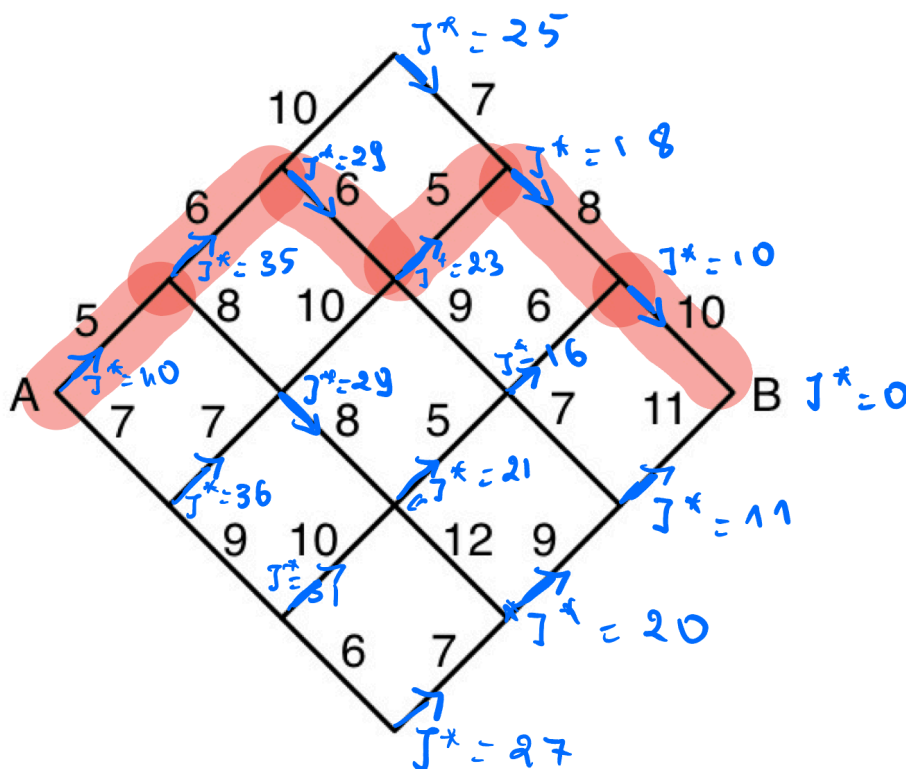


Figure 1: Shortest path problem on a grid.

Dynamic Programming (DP) to find the shortest path from A to B. The grid is a diamond shape with nodes at the intersections. The cost to go from a node to the right and to the left is given by the edge weights. The cost to go from a node to the right and to the left is given by the edge weights. Find the number of computations required by a DP algorithm to find the shortest path from A to B. Find the number of routes that such an algorithm would need to compute the shortest path from A to B.

(b)

Number of nodes. For a grid with n segments per side, the diamond has columns $0, 1, \dots, 2n$.

with the number of nodes per column growing from 1 to $n + 1$ then shrinking back to 1:

$$n_{\text{nodes}} = \sum_{k=0}^n (k+1) + \sum_{k=1}^n k = \frac{(n+1)(n+2)}{2} + \frac{n(n+1)}{2} = (n+1)^2.$$

For $n = 3$: $n_{\text{nodes}} = 16$.

Exhaustive search. Every path from A to B consists of $2n$ steps, each either up or down, and must end at B (same row as A), which requires exactly n ups and n downs. The number of such paths is therefore

$$N_{\text{exhaustive}} = \binom{2n}{n} = \frac{(2n)!}{(n!)^2}.$$

For $n = 3$: $\binom{6}{3} = 20$ routes.

Dynamic programming. The DP backward sweep evaluates the cost-to-go J^* once at every node except the terminal B . At each node the computation is a single min over at most 2 successors, so the total number of DP evaluations equals the number of non-terminal nodes:

$$N_{\text{DP}} = (n+1)^2 - 1 = n^2 + 2n.$$

For $n = 3$: $N_{\text{DP}} = 9 + 6 = 15$ evaluations.

Comparison. $\binom{2n}{n}$ grows exponentially ($\sim 4^n / \sqrt{\pi n}$) while $n^2 + 2n$ grows only polynomially, illustrating the considerable computational advantage of DP over exhaustive search.